

Discrete SOLA

config parameters file
(unique for all steps below)

SOLA_toy.cfg

SOLA.cfg

Pre-processing

Parametrisation: Grid + $\mathbb{V}_j$

G_{ij} matrix

Target kernel : $T_j^{(k)}$

(shape + projection on grid)
with unimodular condition

$$\int T^{(k)}(\mathbf{r}) d^3 \mathbf{r} = \sum_{j=1}^M \mathbb{V}_j T_j^{(k)} = 1$$

Searching for the most sparse line

toy problem

$$G_{ij} \quad \mathbb{V}_j \quad d_i$$

compute_target_toy.py
(use Target.py)

general case

to be done by user
depend on grid and
physical problem :

$$G_{ij} \quad \mathbb{V}_j \quad T_j^{(k)} \quad d_i$$

SOLA_preprocessing.py
(use SOLA_utils.py)
Normalisation + sparsity

Processing

$\mathbf{Q}$ / $\mathbf{Q}^{(\eta)}$ matrix, $\mathbf{c}$ vector, $\mathbf{y}^{(k,\eta)}$ vectors

$$c_i = \sum_{j=1}^M G_{ij} \text{ , } \mathbf{y}^{(k,\eta)} = \begin{cases} \left(T_j^{(k)} \sqrt{\mathbb{V}_j} - c_1^{-1} \frac{1}{\sqrt{\mathbb{V}_j}} G_{j1} \right)_{1 \leq j \leq M} \\ -(c_1^{-1} \eta)_{j=M+1} \end{cases}$$

$$Q_{ii'} = -\frac{c_{i'}}{c_1}G_{1i} + \frac{1}{\sqrt{\mathbb{V}_j}}G_{i'+1i}, \mathbf{Q}^{(\eta)} = \begin{pmatrix} \mathbf{Q} \\ -\eta \ (c_i/c_1)_{2 \leq i \leq N} \end{pmatrix}$$

Launch LSQR with $\mathbf{Q}^{(\eta)} / \mathbf{y}^{(k,\eta)} / \eta$
to get the generalised inverse $x_i^{(k)}$

run_SOLABG.py
(use SOLA_utils.py)

Post processing

Model parameters and uncertainties

$$\hat{\mathbf{m}}_k = \sum_{i=1}^N x_i^{(k)} d_i \quad , \quad \sigma_{\hat{\mathbf{m}}_k}^2 = \sum_{i=1}^N \left(x_i^{(k)} \sigma_{d_i} \right)^2$$

Averaging kernel $A_j^{(k)} = \frac{1}{\mathbb{V}_j} \sum_{i=1}^N x_i^{(k)} G_{ij}$
(or Resolution matrix)

SOLA_postprocessing.py
(use SOLA_utils.py)

Appraisal Plotting

Model parameters and uncertainties $\{\hat{\mathbf{m}}_k, \sigma_{\hat{m}_k}\}$ Averaging kernel $A_j^{(k)}$

plotmodel_toy.py

plotAk_toy.py

plot depends on grid :
to be done by user

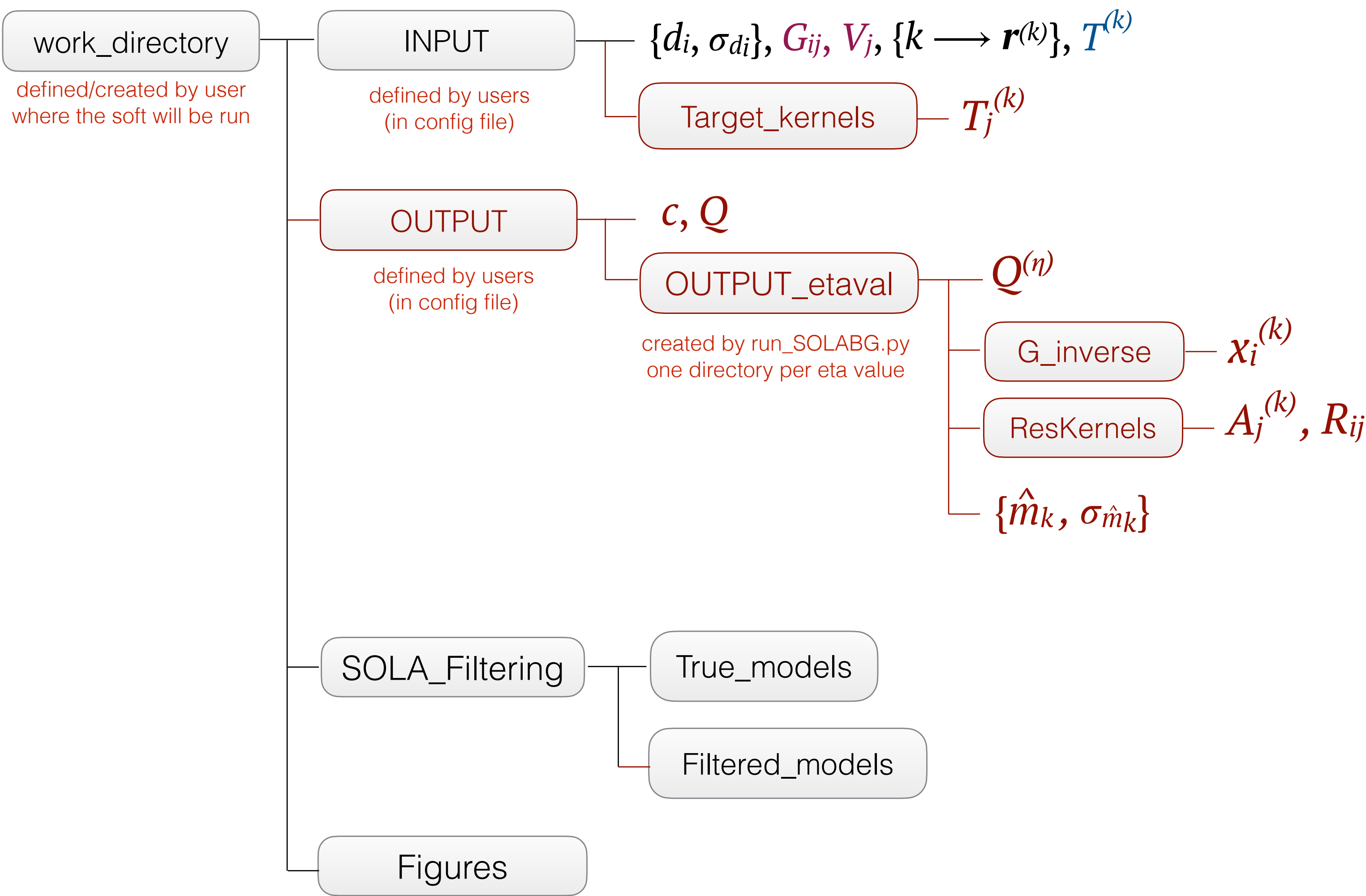
SOLA filtered model

$$\tilde{\mathbf{m}} = \mathcal{R}\mathbf{m} + \mathbf{G}^\dagger \mathbf{n}$$

run_filtering_toy.py

compute_filtmodel.py
plot depends on grid :
to be done by user

Directory architecture



File naming

File name	Mathematical variable
Work_Directory/INPUT/	
Gij_xxxxx.txt	$\mathbf{G} = (G_{ij})$
Gij_xxxxx_normSigma.txt	G_{ij}/σ_{d_i}
Gij_xxxxx_normSigma_sparse0.txt	G_{ij}/σ_{d_i} (most sparse line first)
di_xxxx.txt	$\{d_i, \sigma_{d_i}\}$
di_xxxxx_sparse0.txt	$\{d_i, \sigma_{d_i}\}$ (sorted as G_{ij})
Vj_xxxxx.txt	V_j
xxxxx = radname (in configfile)	
Work_Directory/INPUT/Target_kernels/	
Tj_kval.txt	$T_j^{(k)}$
kval = k value	
Work_Directory/OUTPUT/	
C_vector.txt	c
Q_matrix_com.npz	Q
Work_Directory/OUTPUT/OUTPUT_etaval/	
Q_matrix_etaval.txt	$Q^{(\eta)}$
mk_etaval.txt	$\{\hat{m}_k, \sigma_{\hat{m}_k}\}$
etaval = η value (format : p replace '.' in float number)	
Work_Directory/OUTPUT/OUTPUT_etaval/G_Inverse/	
xk_etaval_kval.txt	$x_i^{(k)}$
Work_Directory/OUTPUT/OUTPUT_etaval/ResKernels/	
Ak_etaval_kval.txt	Averaging kernel : $A_j^{(k)}$
Rij_etaval.txt	Resolution matrix : R_{ij}

INPUT and OUTPUT directories : name of directory can be defined by user (*indir* and *outdir* in configfile)